

Socket Programming in C

TCP and UDP Client-Server Programs for Student Result Calculation

This document contains four C programs demonstrating client-server communication using both TCP (connection-oriented) and UDP (connectionless) sockets. In each pair, a client collects a student's name, PRN (Permanent Registration Number), and marks for 5 subjects, sends this data to the server, and the server computes the percentage and sends it back to the client. Each client also measures the round-trip response time for the request using the **clock()** function.

Program	File	Protocol	Port	Role
1	tcp_server.c	TCP	8080	Server
2	tcp_client.c	TCP	8080	Client
3	udp_server.c	UDP	8081	Server
4	udp_client.c	UDP	8081	Client

1. TCP Server

File: This program implements a TCP (connection-oriented, stream-based) server. It creates a socket, binds it to port 8080, and listens for an incoming client connection. Once a client connects, the server receives a fixed-size **Student** structure containing the name, PRN, and marks for 5 subjects, calculates the percentage, prints the details, and sends the computed percentage back to the client over the same reliable connection.

tcp_server.c

Key Points:

- Uses SOCK_STREAM to create a reliable, connection-oriented TCP socket.
- socket() → bind() → listen() → accept() is the standard TCP server sequence.
- recv()/send() transfer the raw Student struct directly over the connected socket.
- Percentage is computed as the average of the 5 subject marks (each out of 100).
- Handles only a single client connection before terminating.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080

struct Student {
    char name[50];
    char prn[30];
    float marks[5];
};

int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_len = sizeof(client_addr);
    struct Student student;
    float total, percentage;

    // Create socket
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("Socket creation failed");
        exit(1);
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    // Bind
    if (bind(server_fd, (struct sockaddr *)&server_addr,
            sizeof(server_addr)) < 0) {
        perror("Bind failed");
        close(server_fd);
        exit(1);
    }

    // Listen
    listen(server_fd, 5);

    printf("TCP Server started...\n");
    printf("Waiting for client...\n");

    // Accept client
```

```

client_fd = accept(server_fd, (struct sockaddr *)&client_addr,
                   &addr_len);

if (client_fd < 0) {
    perror("Accept failed");
    close(server_fd);
    exit(1);
}

// Receive student data
recv(client_fd, &student, sizeof(student), 0);

printf("\nStudent Details Received:\n");
printf("Name : %s\n", student.name);
printf("PRN  : %s\n", student.prn);

total = 0;

for (int i = 0; i < 5; i++) {
    printf("Subject %d Marks : %.2f\n",
           i + 1, student.marks[i]);
    total += student.marks[i];
}

// Each subject is out of 100
percentage = total / 5.0;

printf("Percentage calculated: %.2f%%\n", percentage);

// Send percentage to client
send(client_fd, &percentage, sizeof(percentage), 0);

close(client_fd);
close(server_fd);

return 0;
}

```

2. TCP Client

File: This program is the TCP client counterpart to `tcp_server.c`. It creates a TCP socket, connects to the server at `127.0.0.1:8080`, and takes student details (name, PRN, and marks for 5 subjects) as input from the user. It then sends this data to the server, waits for the calculated percentage in response, and displays the result along with the round-trip response time measured using `clock()`.

`tcp_client.c`

Key Points:

- `socket()` → `connect()` is the standard TCP client sequence.
- `inet_addr("127.0.0.1")` targets the server running on the same machine (localhost).
- `scanf(" %[\n]", ...)` reads the full name including spaces; `%s` reads the PRN as a single token.
- `clock()` is called immediately before `send()` and after `recv()` to time the request-response round trip.
- `response_time` is converted from clock ticks to milliseconds via `CLOCKS_PER_SEC`.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <time.h>

#define PORT 8080

struct Student {
    char name[50];
    char prn[30];
    float marks[5];
};

int main() {
    int sock;
    struct sockaddr_in server_addr;
    struct Student student;
    float percentage;
    clock_t start, end;
    double response_time;

    // Create socket
    sock = socket(AF_INET, SOCK_STREAM, 0);

    if (sock < 0) {
        perror("Socket creation failed");
        exit(1);
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);

    // Server IP address
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    // Connect to server
    if (connect(sock, (struct sockaddr *)&server_addr,
                sizeof(server_addr)) < 0) {
        perror("Connection failed");
        close(sock);
        exit(1);
    }

    printf("Connected to TCP Server.\n\n");
    // Input student details
```

```

printf("Enter student name: ");
scanf("%s", student.name);

printf("Enter PRN: ");
scanf("%s", student.prn);

for (int i = 0; i < 5; i++) {
    printf("Enter marks for Subject %d: ", i + 1);
    scanf("%f", &student.marks[i]);
}

// Start response-time measurement
start = clock();

// Send data
send(sock, &student, sizeof(student), 0);

// Receive percentage
recv(sock, &percentage, sizeof(percentage), 0);

// End response-time measurement
end = clock();

response_time =
    ((double)(end - start) / CLOCKS_PER_SEC) * 1000;

printf("\n----- Result ----- \n");
printf("Student Name : %s\n", student.name);
printf("PRN          : %s\n", student.prn);
printf("Percentage   : %.2f%%\n", percentage);
printf("Response Time: %.3f ms\n", response_time);

close(sock);

return 0;
}

```

3. UDP Server

File: This program implements a UDP (connectionless, datagram-based) server. Unlike the TCP server, it does not listen or accept connections; it simply binds a datagram socket to port 8081 and waits to receive a Student structure via `recvfrom()`. After computing the percentage, it sends the result back to the same client address using `sendto()`.

udp_server.c

Key Points:

- Uses `SOCK_DGRAM` to create a connectionless UDP socket.
- No `listen()/accept()` step is needed, since UDP has no persistent connection.
- `recvfrom()` captures the sender's address (`client_addr`), which is required to reply.
- `sendto()` sends the reply directly to the captured `client_addr` rather than over a connected socket.
- Runs on port 8081, distinct from the TCP server's port 8080.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8081

struct Student {
    char name[50];
    char prn[30];
    float marks[5];
};

int main() {
    int sock;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_len = sizeof(client_addr);
    struct Student student;
    float total, percentage;

    // Create UDP socket
    sock = socket(AF_INET, SOCK_DGRAM, 0);

    if (sock < 0) {
        perror("Socket creation failed");
        exit(1);
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    // Bind socket
    if (bind(sock, (struct sockaddr *)&server_addr,
            sizeof(server_addr)) < 0) {
        perror("Bind failed");
        close(sock);
        exit(1);
    }

    printf("UDP Server started...\n");
    printf("Waiting for client data...\n");

    // Receive student data
    recvfrom(sock, &student, sizeof(student), 0,
             (struct sockaddr *)&client_addr, &addr_len);
```

```
printf("\nStudent Details Received:\n");
printf("Name : %s\n", student.name);
printf("PRN : %s\n", student.prn);

total = 0;

for (int i = 0; i < 5; i++) {
    printf("Subject %d Marks : %.2f\n",
        i + 1, student.marks[i]);
    total += student.marks[i];
}

// Calculate percentage
percentage = total / 5.0;

printf("Percentage calculated: %.2f%%\n", percentage);

// Send result back to client
sendto(sock, &percentage, sizeof(percentage), 0,
    (struct sockaddr *)&client_addr, addr_len);

close(sock);

return 0;
}
```

4. UDP Client

File: This program is the UDP client counterpart to `udp_server.c`. It creates a datagram socket and, without establishing a connection, takes student details as input and sends them directly to the server at `127.0.0.1:8081` using `sendto()`. It then waits to receive the computed percentage via `recvfrom()` and reports the result along with the measured response time.

`udp_client.c`

Key Points:

- Uses `SOCK_DGRAM`; no `connect()` call is made since UDP is connectionless.
- `sendto()` and `recvfrom()` explicitly specify the destination/source address each time.
- Same input pattern as the TCP client: name, PRN, and 5 subject marks.
- Response time is measured the same way as the TCP client, allowing direct TCP vs UDP comparison.
- Because UDP is unreliable, this simple version has no retry logic if a packet is lost.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <time.h>

#define PORT 8081

struct Student {
    char name[50];
    char prn[30];
    float marks[5];
};

int main() {
    int sock;
    struct sockaddr_in server_addr;
    struct Student student;
    float percentage;
    socklen_t addr_len = sizeof(server_addr);

    clock_t start, end;
    double response_time;

    // Create UDP socket
    sock = socket(AF_INET, SOCK_DGRAM, 0);

    if (sock < 0) {
        perror("Socket creation failed");
        exit(1);
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    printf("UDP Client started.\n\n");

    // Input student details
    printf("Enter student name: ");
    scanf("%[^\n]", student.name);

    printf("Enter PRN: ");
    scanf("%s", student.prn);

    for (int i = 0; i < 5; i++) {
```



```

        printf("Enter marks for Subject %d: ", i + 1);
        scanf("%f", &student.marks[i]);
    }

    // Start response-time measurement
    start = clock();

    // Send data to server
    sendto(sock, &student, sizeof(student), 0,
           (struct sockaddr *)&server_addr,
           sizeof(server_addr));

    // Receive percentage
    recvfrom(sock, &percentage, sizeof(percentage), 0,
             (struct sockaddr *)&server_addr,
             &addr_len);

    // End response-time measurement
    end = clock();

    response_time =
        ((double)(end - start) / CLOCKS_PER_SEC) * 1000;

    printf("\n---- Result ----\n");
    printf("Student Name : %s\n", student.name);
    printf("PRN       : %s\n", student.prn);
    printf("Percentage  : %.2f%%\n", percentage);
    printf("Response Time: %.3f ms\n", response_time);

    close(sock);

    return 0;
}

```

TCP vs UDP: Summary

Both implementations solve the same problem — sending student marks and receiving a computed percentage — but differ in their transport guarantees. TCP is connection-oriented and reliable, requiring the extra `listen()/accept()/connect()` handshake, while UDP is connectionless and faster to set up but offers no delivery guarantee, ordering, or built-in error recovery. Comparing the measured `response_time` from both clients illustrates the typical overhead difference between the two protocols on a local connection.